UF1 – INTRODUCCIÓ A LA PROGRAMACIÓ

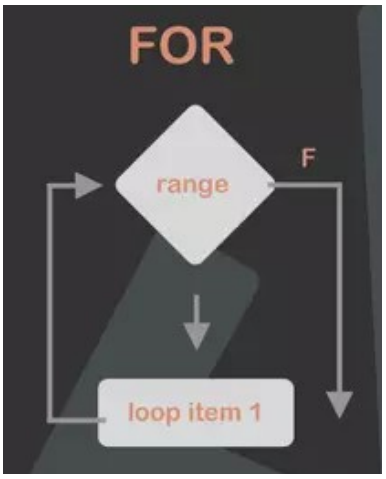
A03 – PROGRAMACIÓ AMB PYTHON

CONTROL DE FLUX AMB BUCLES

El bucle més elemental sol utilitzar-se quan un programa necessita repetir un número de vegades un procés fins que una variable aconsegueix un valor o fins que es compleixi alguna condició predeterminada. Per això, en molts casos, els bucles incorporen variables que actuen com a comptadors que van canviant el seu valor en cada cicle o variables que representen un canvi d'estat (per exemple, Veritable i Fals).

Python subministra dos tipus diferents de bucles: el bucle **for** i el bucle **while**, que corresponen al bucle controlat per:
- la recopilació de valors i/o vegades. Un número predefinit de valors i/o vegades (**for**).
- i el bucle controlat per la condició (**while**)

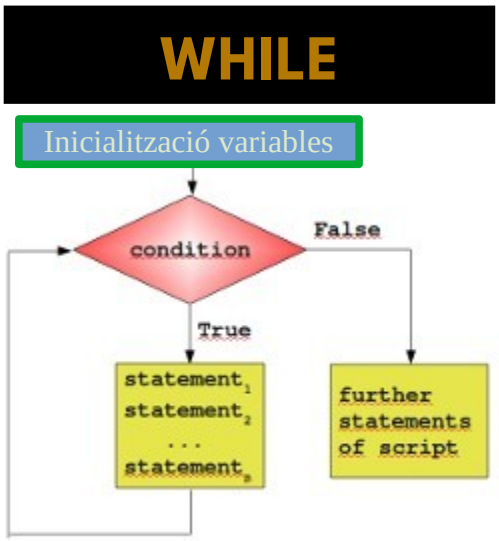
La majoria dels bucles contenen un **comptador** o més, i variables, que canvien els seus valors durant el càlcul.



Aquestes variables **comptador** s'han d'inicialitzar **abans de començar el bucle.**

En un bucle **for**, cal fer un número d'iteracions i/o rang de valors que sabem a priori:
- nº predeterminat de vegades que cal repetir
- nº d'elements d'una llista que sabem

i la variable comptador en un **for** és implícita, forma part de la pròpia estructura del bucle.



La variable condició haurà de ser alterada durant l'execució amb la finalitat d'evitar un bucle infinit.

Abans que s'executi el cos del bucle, s'avalua la condició. Si s'avalua en False, el bucle **while** s'acaba. Això significa que el flux del programa continuarà amb la primera afirmació després de la instrucció **while**, és a dir,

en el mateix nivell de sagnat que el bucle **while**.

Si la condició s'avalua com a vertadera, el cos, - el bloc sagnat dintre del **while** - s'executa. Un cop acabat el cos, es tornarà a avaluar la condició. El cos del bucle s'executarà sempre que la condició produeixi un True

Control del flux amb bucles: for ... in

La construcció utilitza una variable que en cada cicle pren el valor d'un nou element de la llista de la clàusula **in**. El bucle acabarà quan s'arribi a l'últim element de la llista o quan es forci la seva finalització amb **break**.

Sintaxi:

variable_a_avaluar=llista_de_valors

per a tot valor de la variable_a_avaluar que estigui en la **llista_de_valors:**

Cos: Executar totes les instruccions sagnades

Exemple:

7 elements en la llista

```
loteria = [12019, 23023, 90326, 40506, 89450, 21023, 13237]

for numero in loteria:    # numero agafa cada vegada (iteració) un valor (element)
    if numero == 21023:   # consecutiu de la llista.
        print('Has aconseguit el primer premi:', numero)
        break             # Si ha trobat, sortir del bucle for!
```

7 vegades (iteracions)

Amb **for** també es pot fer servir clàusula **in range** que, en el seu ús més elemental, recorre una seqüència de valors començant per 0.

Sintaxi:

per a tot valor de la variable_a_avaluar que estigui en el **rang**:

Cos: Executar totes les instruccions sagnades

Exemple:

```
for num in range(10):
    print(num)   # recorre del 0 al 9 (10 valors amb inici el 0)
                 # per defecte (implícitament) agafa d'1 en 1.
```

range(begin, end, step)

Els valors possibles seran des de el begin(inclòs) fins a end (no inclòs) i els valors aniran incrementant-se segons l'step (que potser positiu o negatiu, però no nul)

Exemple:

```
for i in range(1,11, 1): # Valor inicial:1, Nº de valors: 11 (i agafa de 0 a 10)
                         # Step: 1 (Pas, comptar d'un en un)
    print(i) #imprimeix iterativament números del 0 al 10
```

11 vegades

Control del flux amb bucles: for ... in

Exemples:

```
## Bucle FOR bàsic:

    # Indicant explícitament els paràmetres del range
    for contador in range(0,5,1):  # contador agafa iterativament els valors del 0 al
        print(contador)            # 4, d'un en un
```

La variable contador agafa els valors del rang especificat valor inicial: 0, nº de valors: 5, pas de cada valor: 1. Per tant agafa els cinc valors: 0, 1, 2, 3 i 4
Les instruccions que hi han "dintre" del bucle (identat), es farà per cada valor que agafa la variable "contador". Anem a veure les diferents "iteracions" (repeticions, vegades que es repeteix el codi dintre del "bucle" for):

```
contador=0 (1ª iteració o repetició)
      print(0)   # Això imprimeix "0"

contador=1 (2ª iteració o repetició)
      print(1)   # Això imprimeix "1"

contador=2 (3ª iteració o repetició)
      print(2)   # Això imprimeix "2"

contador=3 (4ª iteració o repetició)
      print(3)   # Això imprimeix "3"

contador=4 (5ª iteració o repetició)
      print(4)   # Això imprimeix "4"
```

El resultat en pantalla seria (alineant números):

```
0
1
2
3
4
```

```
    # Indicant explícitament els paràmetres del range
    for contador in range(0,5): # contador va del valor inicial 0 fins al 4 (5-1),
        print(contador)         # i implícitament d'un en un


    # Indicant explícitament els paràmetres del range
    for contador in range(5): # Per defecte contador agafa 5 valors d'1 en 1,
        print(contador)       # va del valor inicial 0 fins al 4 (5-1),
                              # implícitament agafa d'un en un


## Bucle en el que es van creant 5 números aleatoris (entre 1 i 10) i els mostra per pantalla:

    import random

    for i in range(5):
        x = random.randint(1,10) # Generar un número aleatori del 1 al 10
        print(x)



## Veure si un element està en una llista (array) de valors

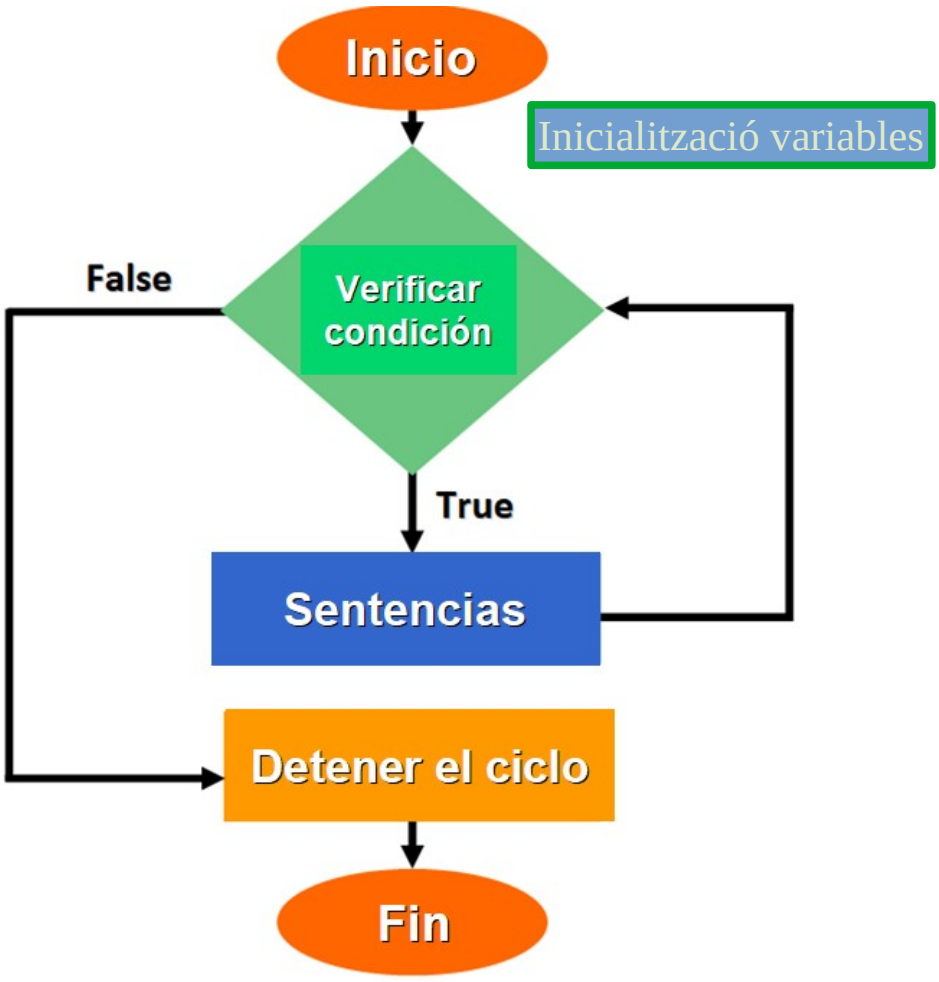
    fruits = ["apple", "banana", "cherry", "orange"]
    for x in fruits:
        print(x)
        if x == "banana":
            print("Nyam, nyam!! Trobada la banana!!")
    print("FINAL!!!")
```

Control del flux amb bucles: while

Bucles: Sentències (instruccions) repetitives, iteratives, que es realitzen més d'una vegada.
De **control de flux**: control del fil d'execució d'un programa.

Per què es fan servir els bucles?
Els fem servir per repetir una seqüència d'instruccions o sentències un nombre indefinit de vegades. Aquest tipus de cicle s'executa mentre una condició donada és True (veritable) i només s'atura si la condició és False (falsa).

Quan escrivim un bucle (cicle) **while**, no definim explícitament quantes iteracions seran completades, només escrivim una **condició** que ha de ser veritable (True) per continuar el procés i falsa (False) per aturar-lo.



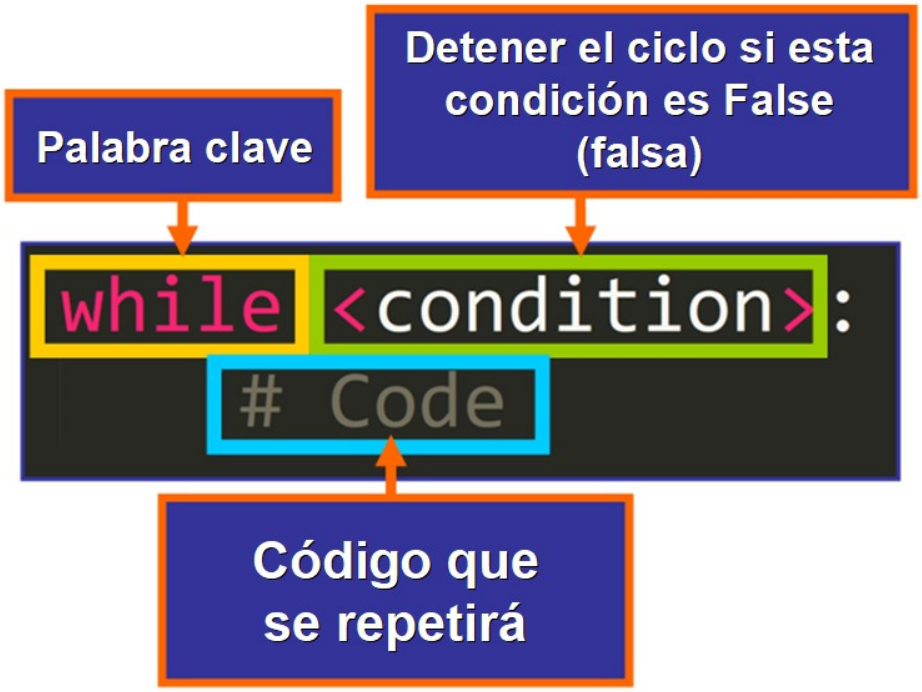
Si la **condició** del bucle while mai és falsa (False), llavors tindrem un **bucle infinit**, el qual és un cicle que en teoria mai s'atura sense que sigui interromput de forma externa. Només podem interrompre amb una drecera de teclat (CTRL + C), a no ser que internament dintre del bucle hi hagi alguna instrucció condicional amb un 'Break'.

Control del flux amb bucles: while

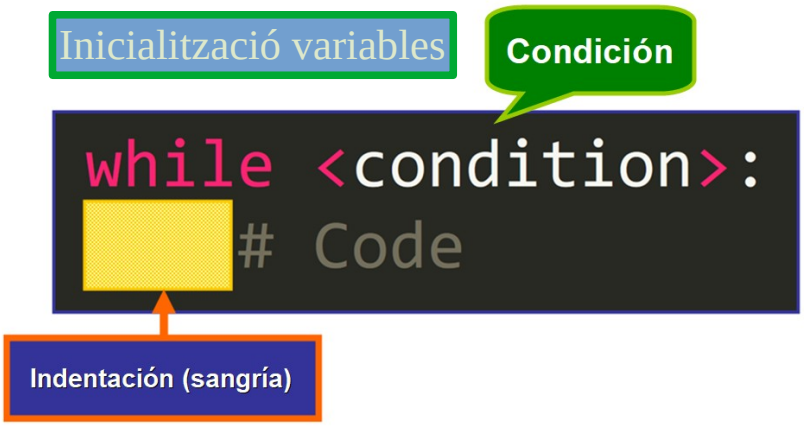
Analitzem aquest procés amb més detall:

- El procés s'inicia quan es troba un cicle while durant l'execució del programa.
- S'avalua la condició per determinar si és vertadera o falsa (Trueo False).
- Si la condició és veritable (True), la seqüència d'instruccions o sentències que pertanyen al cicle while s'executen.
- Novament s'avalua i es verifica la condició del cicle while.
- Si la condició és veritable (True) novament, la seqüència d'instruccions al cos del cicle while s'executen i el procés es repeteix.
- Quan la condició és falsa (False), el cicle s'atura i el programa continua més enllà del cicle while.

Nosaltres **hem d'actualitzar els seus valors explícitament al nostre codi per assegurar-nos que el cicle s'aturarà** en algun moment, quan la condició sigui falsa (False).



Sintaxi general del bucle **while**



Control del flux amb bucles while - EXEMPLE:

Ara que ja saps com funcionen els bucles **while** i com escriure'ls a Python, vegem al detall com funcionen 'per sota' amb alguns exemples.

Exemple 1:

```
# Inicialització variables
i = 4     # Variable comptador (variable de control del bucle => condició)
while i < 8:    # Bucle: repetir fins que i=8
        print(i)    # Instruccions del bucle
        i += 1      # Actualització de la variable de control (variable comptador)
                    # és el mateix que i = i +1
```

Si executem el codi, veiem el resultat següent:

```
4
5
6
7
```

Vegem què passa 'per sota' quan s'executa el codi:

Iteración	i	i < 8	
1	4	True	
2	5	True	
3	6	True	
4	7	True	
5	8	False	Stop

- **Iteració 1**: inicialment, el valor de i és 4, així que la condició avalua a i el bucle inicia la seva execució. El valor es mostra (4) i aquest valor s'incrementa en 1. El cicle torna a iniciar. (i < 8) => True
- **Iteració 2**: ara el valor i és 5, així que la condició avalua a . El cos del cicle s'executa, el valor de es mostra (5) i el valor de s'incrementa a 1. El cicle inicia novament. (i< 8) => True
- **Iteracions 3 i 4**: el mateix procés es repeteix per a la tercera i la quarta iteració, així que els números 6 i 7 es mostren al terminal.
- **Abans d'iniciar la cinquena iteració**, el valor de i és 8. Ara la condició del cicle while i < 8 avalua a False i el bicle while s'atura immediatament.

Si la condició del bucle while és falsa (False) abans d'iniciar la primera iteració, el cicle no s'executa. S'omet i continua amb l'execució del programa principal.

Control del flux amb bucles while - BREAK

La sentència **while** inclou la condició que s'ha de complir perquè s'executi el bloc de codi que conté, encara que en qualsevol moment podem sortir d'un bucle amb **break** o cancel·lar el cicle actual i continuar amb l'execució del cicle següent.

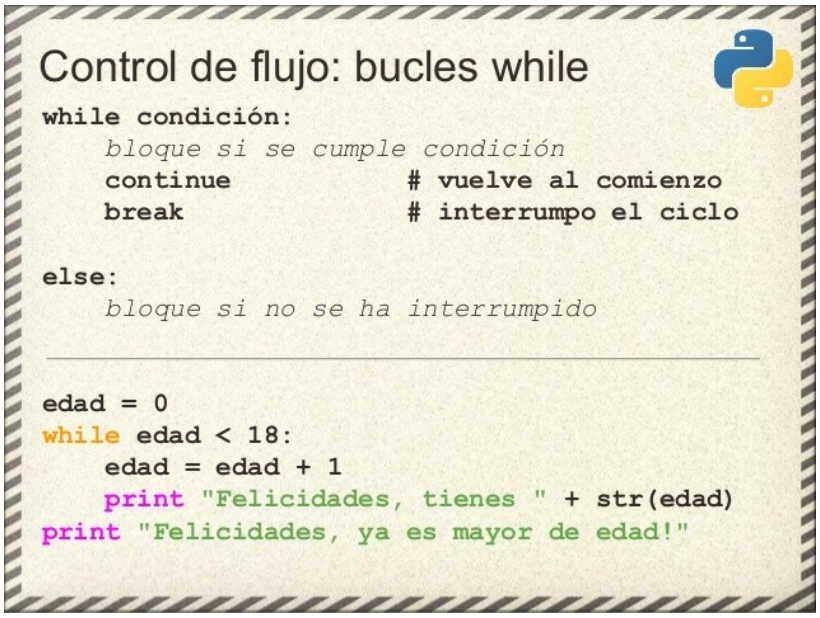
Sintaxi:

Inicialització de la variable_a_ avaluar

mentre la variable_a_avaluar compleixi la condició:

Cos: Executar totes les instruccions sagnades (identades)

variar la variable_a_avaluar.



Exemple 2:

```
Cont = 0     # Inicialització de variables
limite = 5
while cont < 11:  # el bucle termina si cont=10 o superior
    if cont == limite:  # o si arriba al valor límit
        break # Farà que sortir del bucle
    else:
        cont = cont + 1  # cont s'incrementa en 1
        print(cont, limite)
```

Bibliografia i web-grafia

https://docs.python.org/3/tutorial/index.html

https://www.python-course.eu/python3_variables.php

Python 3 al descubierto - 2a ed., Arturo Fernández, Alfaomega Grupo Editor, 30 de set. 2013

http://python-para-impacientes.blogspot.com.es/2014/01/entrada-estandar-input.html

https://www.freecodecamp.org/news/python-while-loop-tutorial/

https://www.mclibre.org/consultar/python/lecciones/python-for.html